Table of Contents

Abstract 3

1. Introduction 4

    1.1 Objectives 4

    1.2 Problem Statement 4

2. Literature Review 5

3. Project Overview and Repository Analysis 6

    3.1 Technology Stack and Rationale 7

    3.2 Dataset Description 7

4. Application Tabs and Functional Benefits 8

5. Methodology 9

    5.1 Pseudocode 10

6. Module-by-Module Code Analysis 10

7. Results and Discussion 12

    7.2 Error Analysis 22

8. Pros, Cons, and Design Tradeoffs 22

9. Conclusion and Future Work 23

    9.1 Future Work 23

References 24

SEM-Based Perovskite Solar Cell Defect Annotation and Detection Using Classical Image Processing and Deep Learning

Abstract

Perovskite solar cells which require attention to microstructure as defects like pinholes, residual PbI2 rich areas, and background morphological variations play a role in charge transport, recombination and device stability. We have developed a Python based annotation, preprocessing and object detection workflow for perovskite thin film Scanning Electron Microscopy (SEM) images in this project. The system puts together an interactive Webapp annotation interface, YOLO format label generation, OpenCV and Segment Anything Model (SAM) supported auto annotation, patch-based data set preparation, and model testing across YOLOv8, YOLO11, and Faster R-CNN pipelines. PbI-2, 3D pinholes, 3D-2D pinholes, 3D background, and 3D-2D background. We report a division of 4,701 image patches into 3,759 for training, 467 for validation, and 474 for testing. We have put up 6 YOLO training runs in the repo. Out of which YOLO11m did the best with mAP50 at 0.4219 and mAP50-95 at 0.1905. We see that real time object detectors are useful for fast SEM defect localization but at the same time the moderate mAP scores also bring to light that small, low contrast, texture similar defects still present a challenge. We present a reproducible end to end toolchain for lab annotation, model training, evaluation and also a path forward towards more robust microscopy defect analysis.

1. Introduction

Scanning electron microscopy is a key tool in the characterization of thin film photovoltaic materials as it presents grain morphology, pinholes, phase segregation, surface coverage and other microstructural features which are hard to determine from electrical measurements alone. In perovskite solar cells SEM is used by researchers to determine film quality, to identify fabrication issues, and to compare the results of different additives or processing methods. But manual inspection is slow, subjective and hard to scale when researchers have to go over hundreds or thousands of images across many material and processing variables.

The SEM-Annotation project fills in this void by turning manual image inspection into a practical annotation and model training application. We have a Webapp app (main.py) which is the interface, we have modular handlers for both manual and automatic annotation, we have utility scripts for patch generation and dataset split, classical image processing filters, YOLO and Faster R-CNN training logic, and we also have saved results of experiments. The project is not a single model but a full out solution which takes microscopy images and turns them into labeled data, trains detectors, compares models and feeds predictions back into the annotation loop.

The issue at hand is to identify and determine the location of perovskite defects in SEM images at the same time that we keep the workflow accessible for researchers which may not be deep learning experts [1]. We have taken a practical hybrid approach to this. We use manual annotation which gives us human input and ground truth. OpenCV methods we have used for transparent, parameterized detection of bright PbI 2 like particles and dark pinhole like regions. We also have included SAM provides for rapid table or automatic segmentation when available. YOLO models we have included for rapid localization and auto labeling after the models are trained. Also we have included Faster RCNN as a two stage baseline for comparison.

The scope of the project covers annotation, dataset organization preprocessing training benchmarking visualization. This project does not claim the finished industrial grade defect detector. It provides a solid applied base: a working app, generated labels, trained model artifacts and enough modifiability to enhance dataset and models iteratively.

1.1 Objectives

- Build out a user-friendly SEM image annotation interface for defect bounding boxes and polygon like labels.
- Also support auto and semi auto annotation using YOLO inference, OpenCV threshold/morphology methods, and SAM when the checkpoint is available.
- Also prepare out which are compatible with YOLO and which include train/validation/test split files and class metadata.
- Also train and compare many object detection models using mAP50, mAP50-95, precision, recall, F1 score, and also training time.

1.2 Problem Statement

Current manual approach to labeling perovskite solar cell defects in SEM images is a bottleneck of high throughput materials discovery. While conventional image processing algorithms offer some automation, they tend not to generalize well to the large variance and noise present in microscopic defects. In addition, they are not robust to the multi-scale nature of the perovskite solar cell images. A comprehensive deep learning optimized approach is needed which can unify manual label annotation quality with breakthrough automated detection speed.

2. Literature Review

Although deep networks are learning extremely complex textures at the same time they must be accurately labeled and trained in a huge number of examples [1], also classical filters are very commonly used for interpretation of bright/dark features.

Table 1: Evaluation of Classical Image Processing Techniques and Literature Baselines

Study	Method / Domain	Key Finding	Relation to This Project	Observed Gap
Pan et al., 2025	Deep SEM segmentation for perovskite materials	Identifies lead iodide, perovskite, and defect domains; highlights manual SEM bottleneck.	This project localizes classes with YOLO boxes and adds app-based annotation.	Needs segmentation-grade labels and validation across more morphologies.
Scientific Reports FAPbI3 framework, 2025	Multi-model SEM defect detection and Webapp deployment	Combines preprocessing, deep learning, and practical app deployment.	Very similar motivation; this project adds explicit YOLO benchmarking artifacts and OpenCV/SAM assisted annotation.	Current repo needs stronger model-comparison UI linkage and per-class analysis.
Dey et al., 2022	Mask R-CNN defect detection in SEM images	Shows instance segmentation can improve defect extent estimation.	This project uses Faster R-CNN and YOLO; Mask R-CNN could be a future extension.	Instance masks would improve pinhole shape modeling beyond bounding boxes.
SEM defect ML/DL review, 2021	Survey of SEM defect classification and detection	Summarizes CNN and deep-learning approaches for semiconductor SEM inspection.	Supports computer vision for SEM defect workflows.	Perovskite-specific datasets remain limited.
Redmon et al., 2025	YOLO real-time object detection	Frames detection as single-pass regression, enabling fast inference.	Justifies YOLO for interactive auto-labeling.	YOLO can struggle with small, dense defects and localization precision.
Ren et al., 2025	Faster R-CNN with RPN	Two-stage detector improves proposal quality using shared convolutional features.	Used as a comparison baseline through torchvision Faster R-CNN.	Training and inference cost are higher than YOLO.
Ronneberger et al., 2015	U-Net biomedical segmentation	Effective segmentation with few biomedical images.	Relevant future path for pixel-level SEM defect masks.	Requires segmentation masks rather than current box labels.

Kirillov et al., 2023	Segment Anything	Promptable foundation segmentation model trained on very large mask data.	Repository includes SAM handler for auto-annotation.	Default SAM masks need material-specific filtering and class assignment.
Canny, 1986; OpenCV docs	Edge detection	Formalizes low-error, well-localized edge detection.	Canny is available in OpenCV auto-annotation.	Edges alone over-detect grain boundaries in textured SEM images.
Otsu, morphology, CLAHE literature	Classical thresholding and contrast enhancement	Contrast and morphology remain strong for bright/dark blob extraction.	Used in OpenCV handler and Perovskite Defect Filter.	Sensitive to illumination, scale, and threshold choices.

The hole the implementation is targeted at is a practical integration. While several papers offer a model or an algorithm, a laboratory flow requires annotation preprocessing data set partitioning, training orchestration, graphing of metrics and iterative correction. This project connects those steps in one repository.

3. Project Overview and Repository Analysis

The repository we have is organized around a Webapp entry point and a modular src package. In main.py which is the main entry point we have five app modes put forth via the sidebar. Also in src/model_handler.py we have YOLO model's which we load, train, we get out of it training validation metrics, we prepare the data sets and do auto-label inference. In the src/handlers we have what we use for user interface and auto annotation. In the src/filters we have what we use for image processing which includes an adaptive kernel engine and also a dedicated Perovskite Defect Filter. The scripts folder we have standalone utilities for splitting data sets, training all models, creating benchmark PDFs, image validation and running classical filter experiments.[4]

3.1 Technology Stack and Rationale

Python 3.10, Webapp, Ultralytics YOLO, Torch / Torchvision, OpenCV, Pillow / NumPy / Pandas, Webapp-drawable-canvas, YAML / JSON

Table 2: Integrated Technology Stack and Rationale for Modular Development.

Technology	Role	Why Used	Benefits	Limitations
Python 3.10	Main language	Scientific prototyping and ecosystem compatibility.	Broad library support; simple scripts; reproducible.	Dependency conflicts can occur across CUDA/PyTorch versions.
Webapp	Interactive app UI	Rapid dashboard and annotation workflow without custom frontend.	Simple to run with Webapp run main.py; good for lab demos.	Complex state, drawing tools, and large uploads need more engineering.
Ultralytics YOLO	Object detection	Fast one-stage detector for auto-labeling loops.	Easy training API; stores curves, weights, and metrics.	Small SEM defects can be difficult; metrics are moderate.
Torch / Torchvision	Deep-learning backend and Faster R-CNN	Supports GPU training and two-stage baseline.	Flexible custom dataset; strong pretrained backbones.	Longer training and more code than YOLO API.
OpenCV	Classical image processing	Thresholding, morphology, Canny, CLAHE, contours.	Transparent, parameterized detection for bright/dark features.	Requires tuning per image class and microscope condition.
Pillow / NumPy / Pandas	Image I/O, array math, data tables	Standard scientific Python utilities.	Reliable data conversion and metric manipulation.	Large images can be memory-heavy without batching.
Webapp-drawable-canvas	Manual drawing UI	Rectangles, free drawing, and polygon-like drawing modes.	Enables human-in-loop annotation.	Current code does not yet reload YOLO labels back to canvas objects.
YAML / JSON	Dataset and metric metadata	YOLO config and model metrics are easy to inspect.	Human-readable and script-friendly.	Path consistency must be maintained carefully.

3.2 Dataset Description

The repository contains balanced_dataset with 440 images and 440 label files, plus split files containing 352 train, 44 validation, and 44 test entries. The model-training split used in the current benchmark is dataset_split_224, containing 4701 image patches and 4701 YOLO label files. Its split files contain 3759

training, 467 validation, and 474 testing samples. The patch size is 224 x 224 pixels, chosen to reduce GPU memory use and make training possible on limited VRAM. [5]

Table 3: Dataset Distribution and Class Frequencies Across Processed Patches.

Class ID	Class Name	Object Instances in dataset_split_224 labels
0	PbI2	2910
1	3D_pinholes	3995
2	3D-2D_pinholes	3661
3	3D_background	0
4	3D-2D_background	0

Background classes are represented mainly by images with empty labels. The split contains 1811 non-empty label files. Empty-label background patches are useful because they teach YOLO what not to predict, but the ratio must be controlled so that the detector does not become overly conservative.

4. Application Tabs and Functional Benefits

Tabs in Web App which includes: Data Explorer & Labeling, Train Model, Auto-Annotation Inference, Multi-Model Benchmark, Model Comparison

Table 4: Overview of Functional Modules and Operational Benefits of the Web Application.

Tab	Main Functions	Benefit	Current Limitation
Data Explorer & Labeling	Browse class folders, track progress, jump to first unlabeled image, draw labels, run SAM/OpenCV annotation, save YOLO labels.	Supports human-in-loop ground truth creation.	Default path is machine-specific; existing labels are not rendered back to canvas.
Train Model	Collect image-label pairs, write YOLO dataset YAML, train selected YOLO model.	Allows small training runs from UI.	Long training can block Webapp; scripts are safer for full runs.
Auto-Annotation Inference	Use trained YOLO or SAM to auto-label selected class folders.	Accelerates labeling after an initial model exists.	YOLO confidence must be tuned; SAM checkpoint must be separate.
Multi-Model Benchmark	Train queued models with shared settings and save metrics.	Supports reproducible comparison.	App queue currently differs from broader script training queue.
Model Comparison	Load JSON metrics, render table/chart, identify best model.	Turns outputs into decision support.	Live run exposed path mismatch: app searches runs/metrics, while metrics exist in src/runs/metrics.

5. Methodology

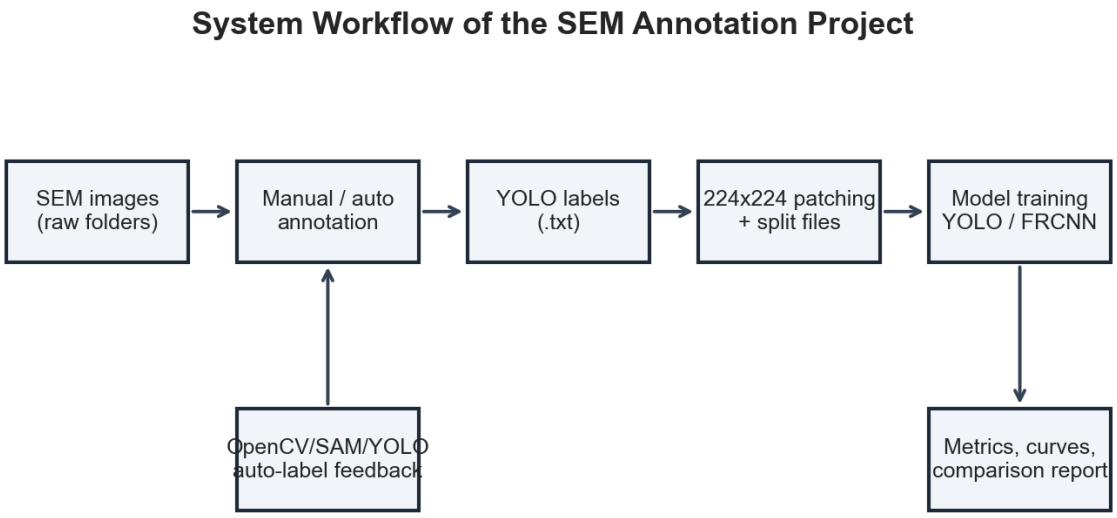


Figure 1. End-to-end workflow implemented by the SEM-Annotation repository.

In the annotation process we begin with a class folder which is chosen in the Data Explorer tab. The application goes through images via utility functions which also check for the presence of corresponding YOLO.txt files and reports progress. As a user you will do the annotation work on a drawable canvas over the image. We see conversion of the drawn rectangles to YOL square center points, width and height. Also free draw or polygon paths are turned into a series of normalized coordinates as we get more points.[6]

The quick auto-annotation which we present includes two modes. OpenCV which we use for the very fast and easy to interpret detection of bright or dark features. SAM which we made available for segment anything out that is when you have the segment anything and the check point file installed. YOLO inference is made available in the Auto Annotation tab which is a trained model that puts out bounding boxes and also produces YOLO label files for images which aren't already annotated [8].

PbI2 rich particles are what you will see to be bright, in the other hand pinholes will be dark. OpenCV which we use to handle the images has support for thresholding, bright thresholding, top-hat transforms, HSV brightness masks, adaptive thresholding, Canny edge detection, watershed style preprocessing, CLAHE, also we have brightness/contrast adjustment and we can do contour filtering by area [7].

The split out into smaller patches which we create from large scale SEM images. We transform bounding boxes from YOLO's normalized coordinates to absolute, clip them to the patch borders, and then we put them back into normalized coordinates for the patch space. This we did to preserve labels through the process of patch generation which also allows us to train on memory constrained hardware [9].

YOLO training is done on Ultralytics. ModelHandler.train_model which loads weights, chooses GPU if present, trains the model, loads best.pt, validates it, and puts out JSON reports of mAP50, mAP50-95, precision, recall, F1, epochs, image size, train time, and weight path. For Faster R-CNN we use torchvision with ResNet-50 FPN backbone and a custom dataset that we designed to transform YOLO labels into torchvision target dictionaries [16].

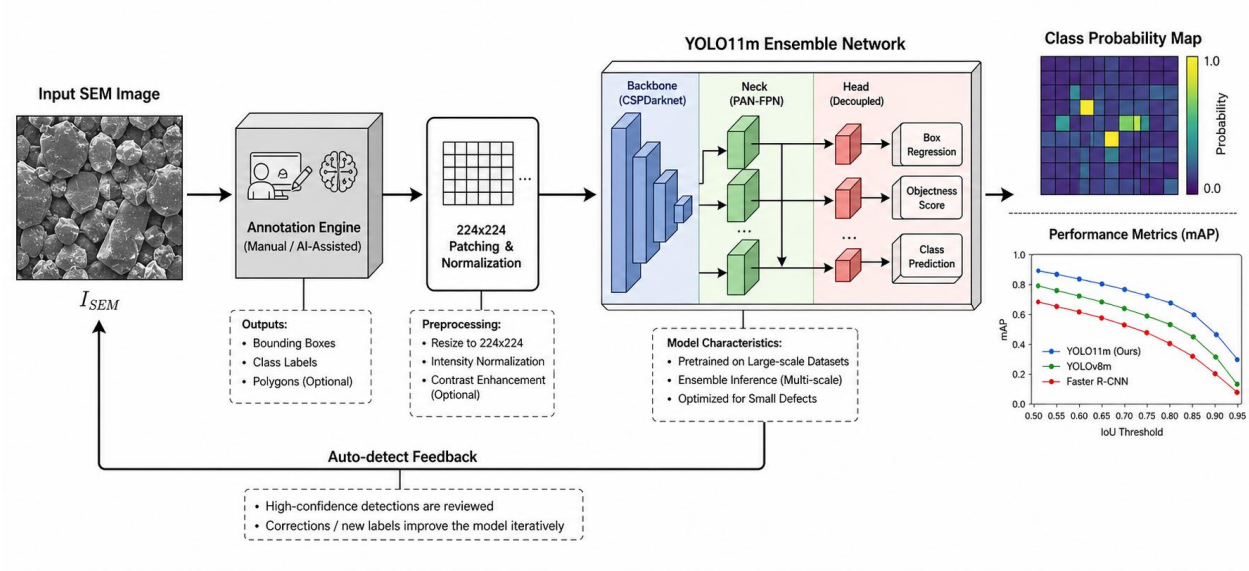


Figure 2. Defect Detection Framework

5.1 Pseudocode

Algorithm 1: Patch-based YOLO dataset preparation
Input: SEM image folder, YOLO label folder, patch size P, overlap O.
1. For each image, read the corresponding YOLO label file.
2. Convert each normalized label into absolute corner coordinates.
3. Generate sliding-window patch coordinates with stride P x (1 - O).
4. Clip intersecting boxes to patch boundaries.
5. Discard boxes whose visible ratio is below the threshold.
6. Convert clipped boxes back to normalized YOLO patch coordinates.
7. Save patch image and matching label file.
8. Write data. yaml and train/val/test index files.

Algorithm 2: Hybrid auto-annotation
Input: List of images and directory for output labels.
1. Select method based on defect class or user configuration.
2. For OpenCV, convert to grayscale and optionally apply CLAHE and brightness/contrast adjustment.
3. Apply threshold, top-hat, color mask, adaptive threshold, Canny, or watershed preprocessing.
4. Extract contours and filter by area.
5. Convert valid contours into YOLO bounding boxes and write label files.
6. For YOLO, run model prediction, extract normalized boxes, and write labels.
7. For the SAM go through the masks, take the predicted IoU from the filters, cast the extent of the masks to boxes and write labels.

6. Module-by-Module Code Analysis

The tradeoff is that several paths are hard-coded to the original Windows machine, and some UI paths diverge from training output paths.

Table 5: Detailed Repository Mapping and Core Component Functionalities.

File	Purpose	Benefit	Limitation / Improvement
main.py	entry point; defines sidebar navigation and all five app modes.	Integrates labeling, training, auto-annotation, benchmarking, and comparison.	Single-file UI is easy to run but large; future refactor could split tabs.
src/model_handler.py	YOLO loading, training, validation, metrics, dataset preparation, and auto-labeling.	Centralizes model logic.	Metric output path differs from current app comparison path.
src/logger_setup.py	Redirects stdout/stderr to app.log.	Preserves app logs.	File handle management could be context-aware.
src/handlers/canvas_ui.py	Drawable canvas over SEM images with class and drawing controls.	Manual annotation UI.	TODO remains for loading existing YOLO labels.
src/handlers/opencv_handler.py	Parameterized contour-based auto-annotation.	Fast, explainable baseline.	Thresholds require tuning.
src/handlers/sam_handler.py	SAM automatic mask generation and conversion to YOLO boxes.	Segmentation-assisted labeling.	Requires checkpoint; class assignment is defaulted.
src/handlers/utils.py	Directory and image-file listing helpers.	Reusable file scanning.	Only scans direct files.
src/utils/annotation_utils.py	YOLO read/write, coordinate conversion, clipping, and pair collection.	Essential for patching.	Could validate malformed labels more strictly.
src/datasets/frcnn_dataset.py	Torch Dataset that reads YOLO labels and returns Faster R-CNN targets.	Bridges YOLO labels to torchvision.	No dedicated augmentation pipeline.
src/filters/adaptive_kernel.py	Variance, gradient, entropy, adaptive kernel maps, hybrid responses.	Adapts filter scale to local texture.	Heavier than simple thresholds.

src/filters/classical_blend.py	CLAHE, top-hat, black-hat, LoG, DoG, Gabor, Hessian, LBP, DCT, entropy.	Library of interpretable filters.	Some functions are expensive and not fully wired into app.
src/filters/perovskite_defect_filter.py	Mode-aware classical detector with dataclasses and NMS.	Domain-specific no-training baseline.	Needs quantitative validation against ground truth.
scripts/split_dataset.py	Sliding-window patch split with box clipping.	Preserves labels during patching.	Can overwrite output directory.
scripts/split_and_train_pipeline.py	Patching, balancing, splitting, and starting training.	Automates preprocessing.	String-replaces training script path, which is brittle.
scripts/train_all_gpu.py	YOLO11 and Faster R-CNN training orchestration.	Reproducible batch training.	Large monolithic script; hard-coded CUDA assumptions.
scripts/generate_benchmark_report.py / create_benchmark_pdf.py	PDF report generation from JSON metrics.	Automated charts and summaries.	Separate from manuscript report.
scripts/dynamic_kernel_detector.py / demo_pdf_filter.py	Run classical detector experiments and save overlays.	Qualitative visualizations.	Classical detections need threshold tuning.

7. Results and Discussion

The strongest recorded model is yolo11m, with mAP50 = 0.4219, mAP50-95 = 0.1905, precision = 0.4488, recall = 0.4820, and F1 = 0.4648. The result suggests that YOLO11m currently gives the best balance of localization quality and training cost in the available experiments.

Table 6: Quantitative Performance Comparison of Model Variants via YOLO Architecture.

Model	**mAP50**	**mAP50-95**	**Precision**	**Recall**	**F1**	**Epochs**	**Img Size**	**Train Time (h)**
yolo11m	0.4219	0.1905	0.4488	0.4820	0.4648	50	224	1.14
yolo11s	0.3957	0.1734	0.4326	0.4582	0.4450	50	224	0.62
yolov8s	0.3779	0.1629	0.4553	0.4249	0.4396	50	224	0.46
yolo11l	0.3526	0.1602	0.4054	0.4315	0.4180	50	224	2.95
yolov8m	0.3528	0.1535	0.3976	0.4300	0.4132	50	224	0.65
yolov8l	0.2374	0.0891	0.3040	0.3083	0.3061	50	224	1.17

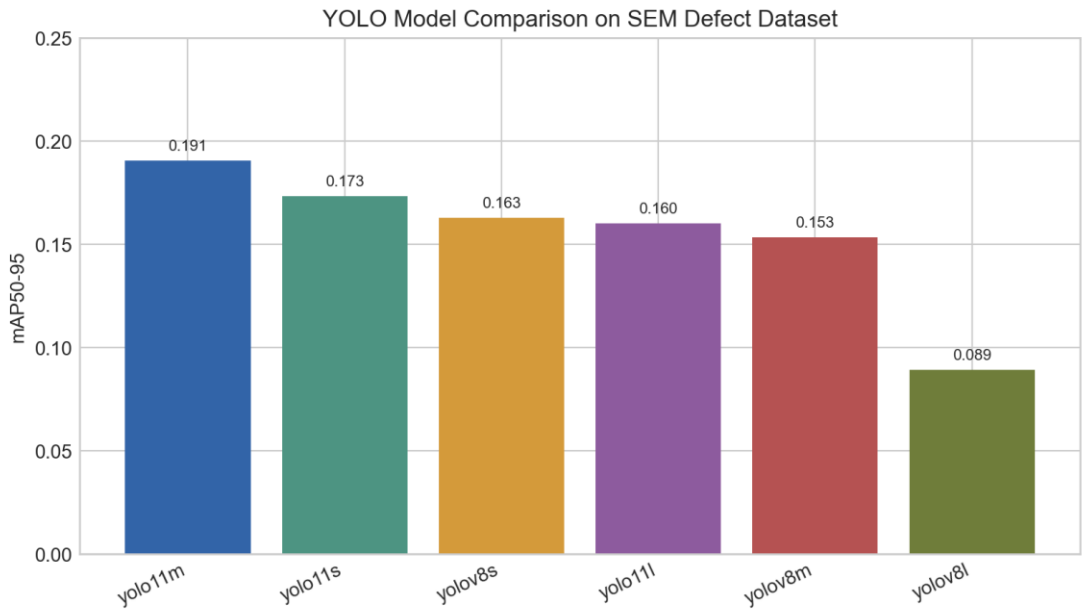


Figure 3. Model Map5095 Comparison.

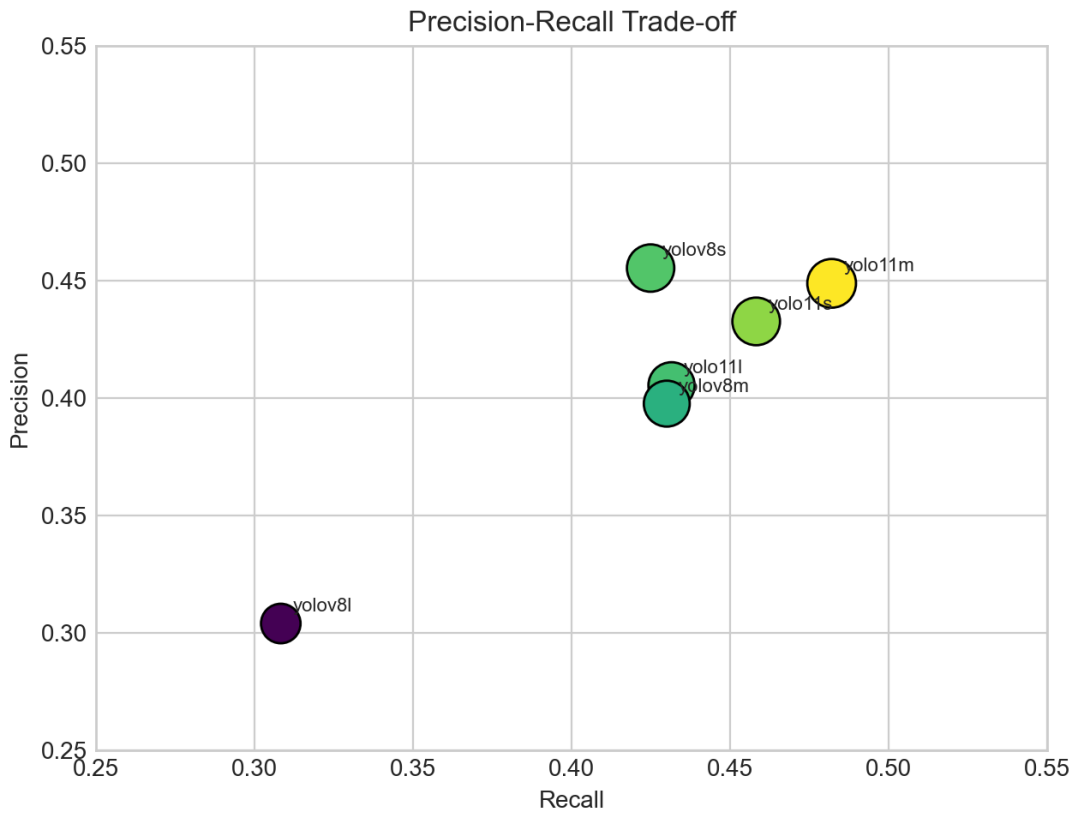


Figure 4. Precision Recall Scatter.

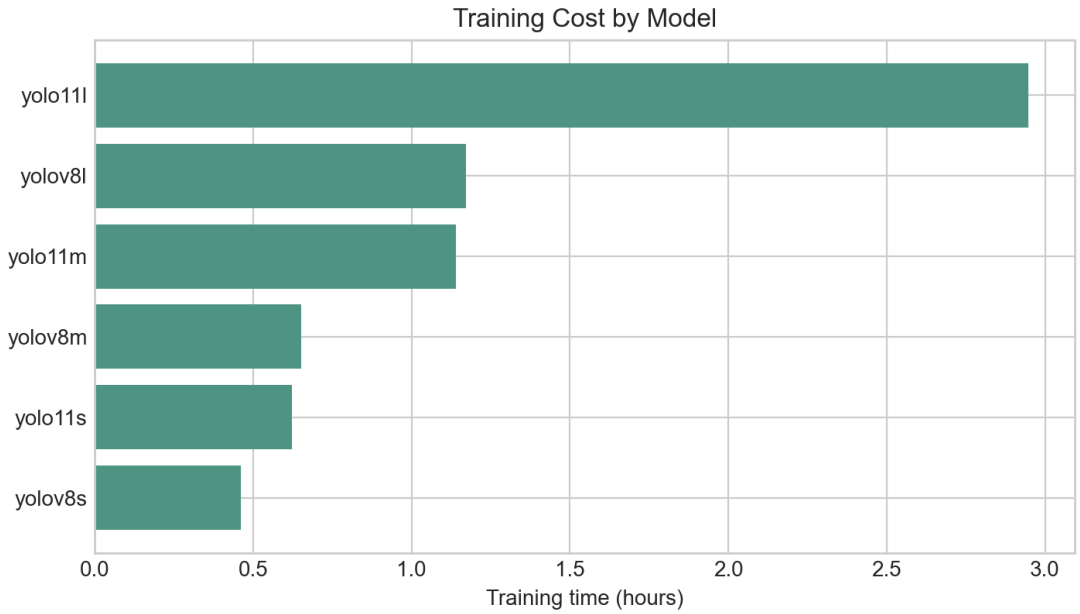


Figure 5. Training Time

The quantitative results report moderate to high scores. This is to be expected for small SEM defects which may only take up a few pixels in a 224 x 224 image also what we see is that the grain texture which surrounds the defect often looks like the target class. Also we see that mAP50 is greater than mAP50-95 as the former is more lenient in what it measures it basically pays less attention to box localization. The more strict mAP50-95 instead reports on boundary errors which we see to be a issue when defects are irregular or only partially present after we apply our patch.7.1 Qualitative Visualizations [12]

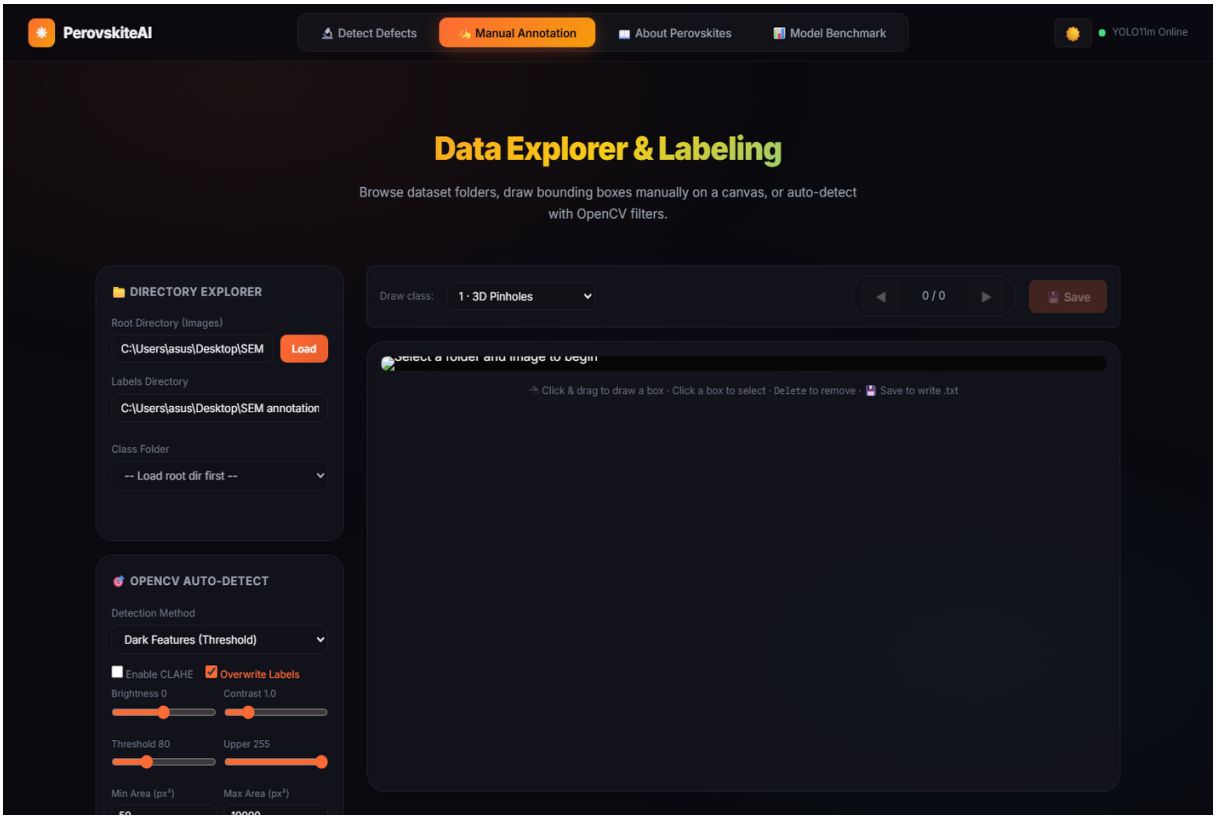


Figure 6.1 .Live Data Explorer & Labeling tab from Webapp run main.py.

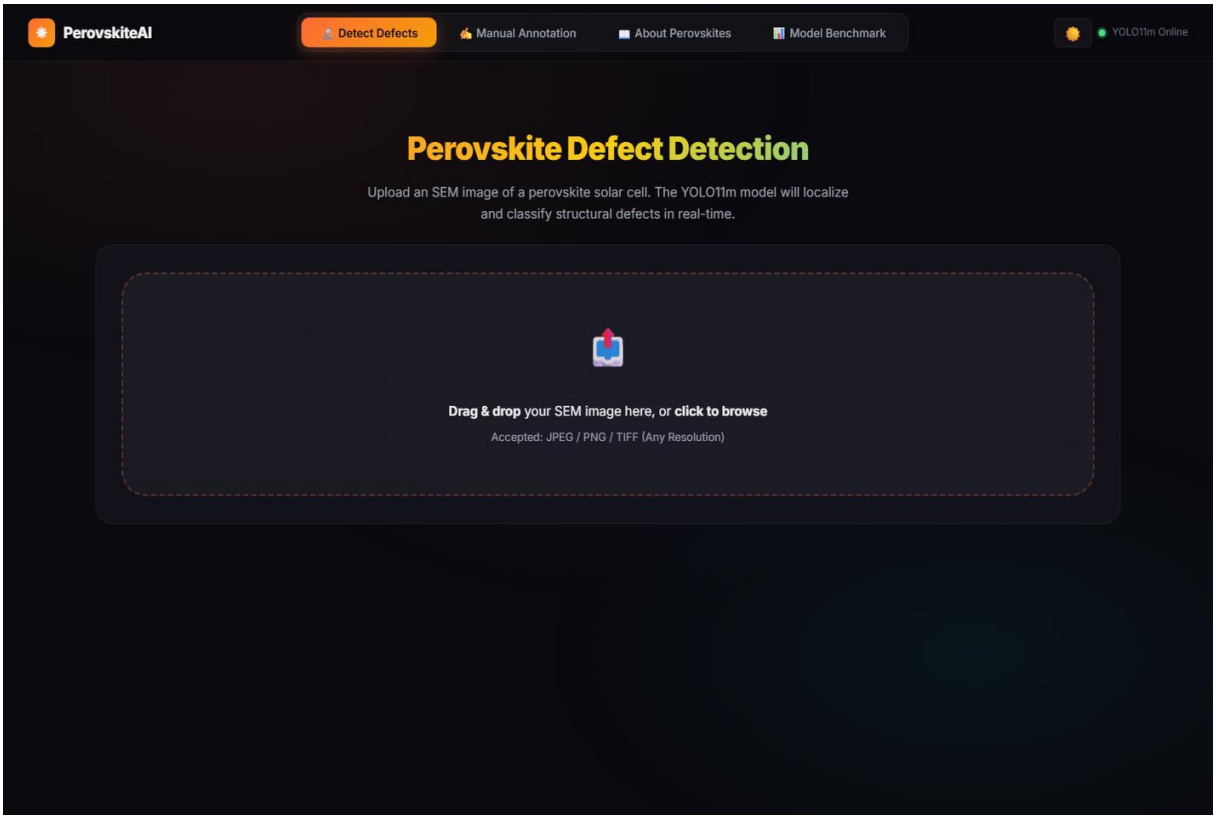


Figure 6.2. Live Train Model tab.

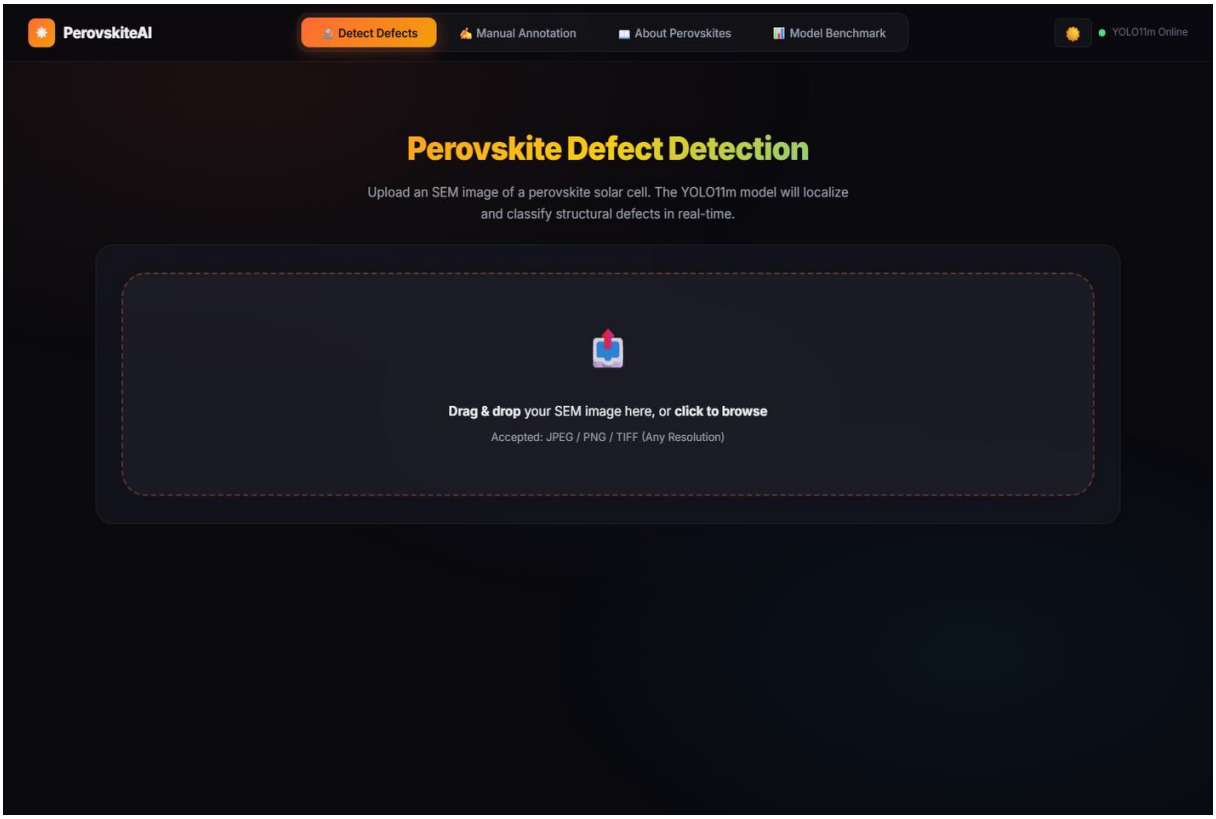
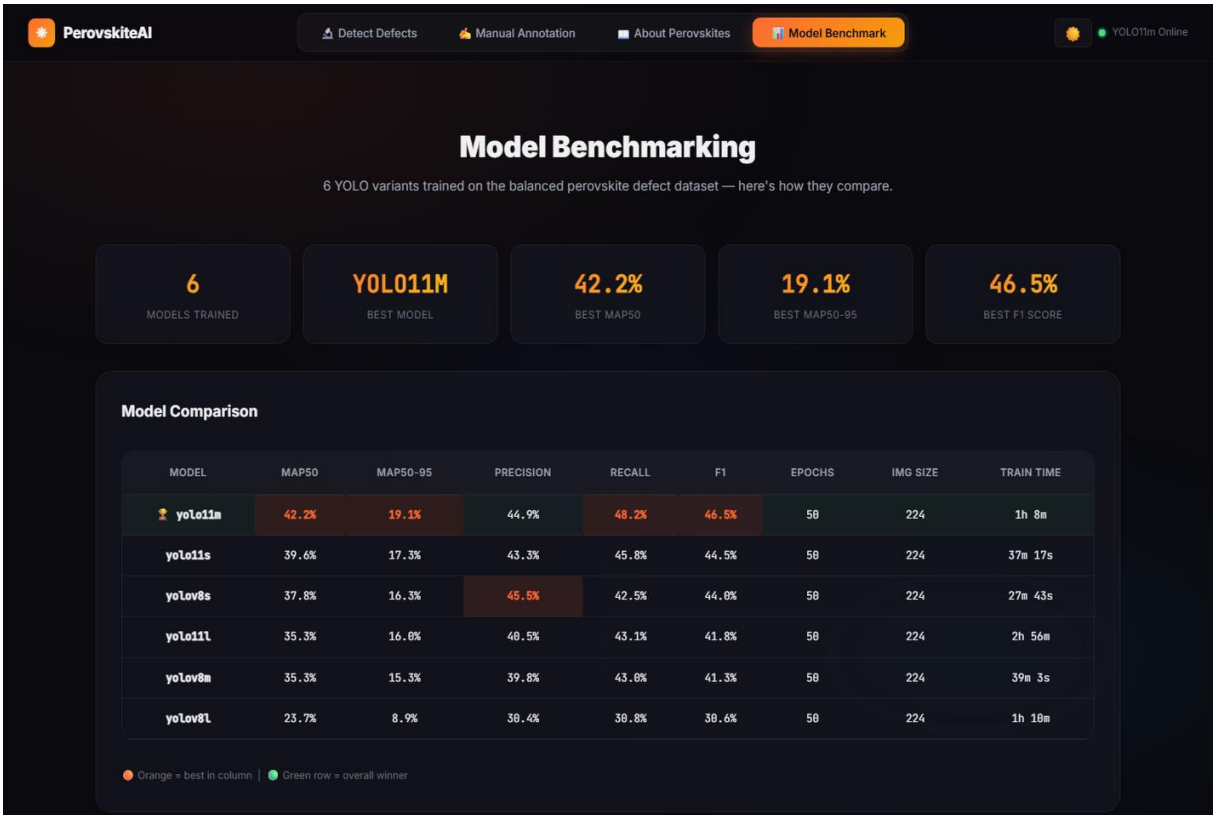


Figure 6.3. Live Auto-Annotation Inference tab.



MODEL	MAP50	MAP50-95	PRECISION	RECALL	F1	EPOCHS	IMG SIZE	TRAIN TIME
yolo11m	42.2%	19.1%	44.9%	48.2%	46.5%	50	224	1h 8m
yolo11s	39.6%	17.3%	43.3%	45.8%	44.5%	50	224	37m 17s
yolov8s	37.8%	16.3%	45.5%	42.5%	44.0%	50	224	27m 43s
yolo11l	35.3%	16.0%	40.5%	43.1%	41.8%	50	224	2h 56m
yolov8m	35.3%	15.3%	39.8%	43.0%	41.3%	50	224	39m 3s
yolov8l	23.7%	8.9%	30.4%	30.8%	30.6%	50	224	1h 10m

Figure 6.4. Live Multi-Model Benchmark tab.

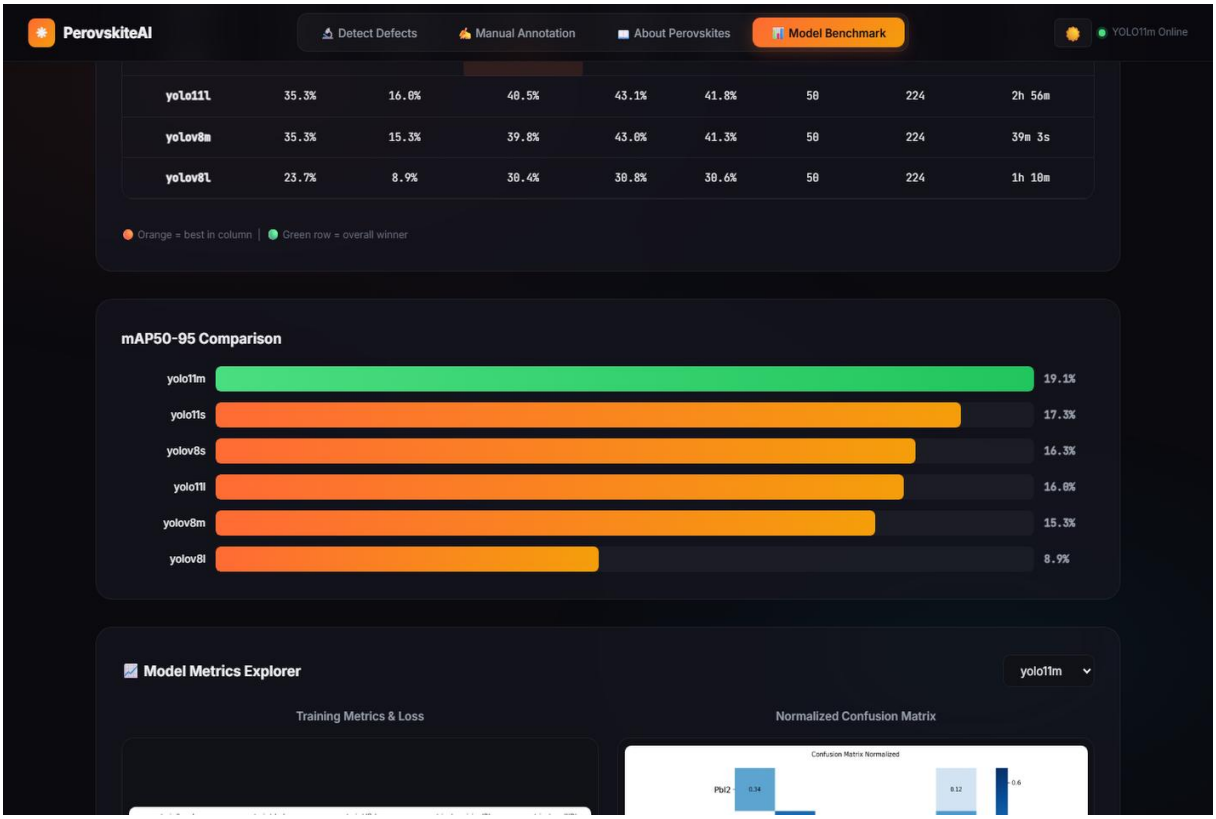


Figure 6.5. Live Model Comparison tab showing the current metric path mismatch limitation.

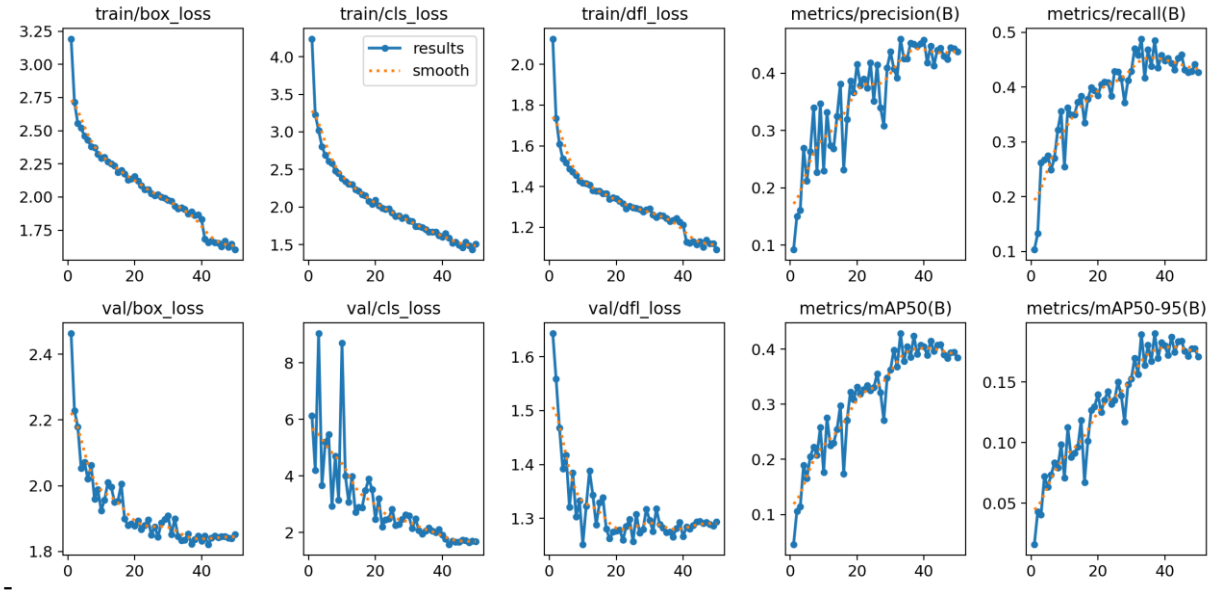


Figure 7. YOLO11m training curves generated by Ultralytics.

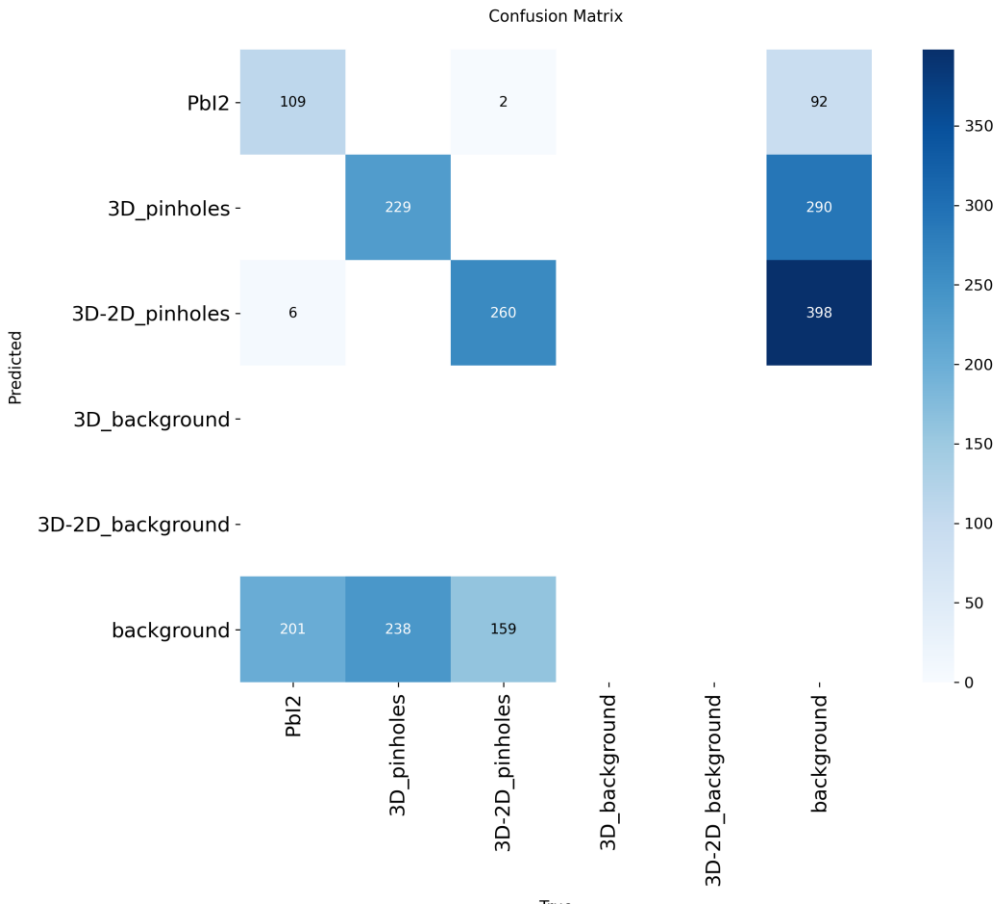


Figure 8. YOLO11m confusion matrix artifact.

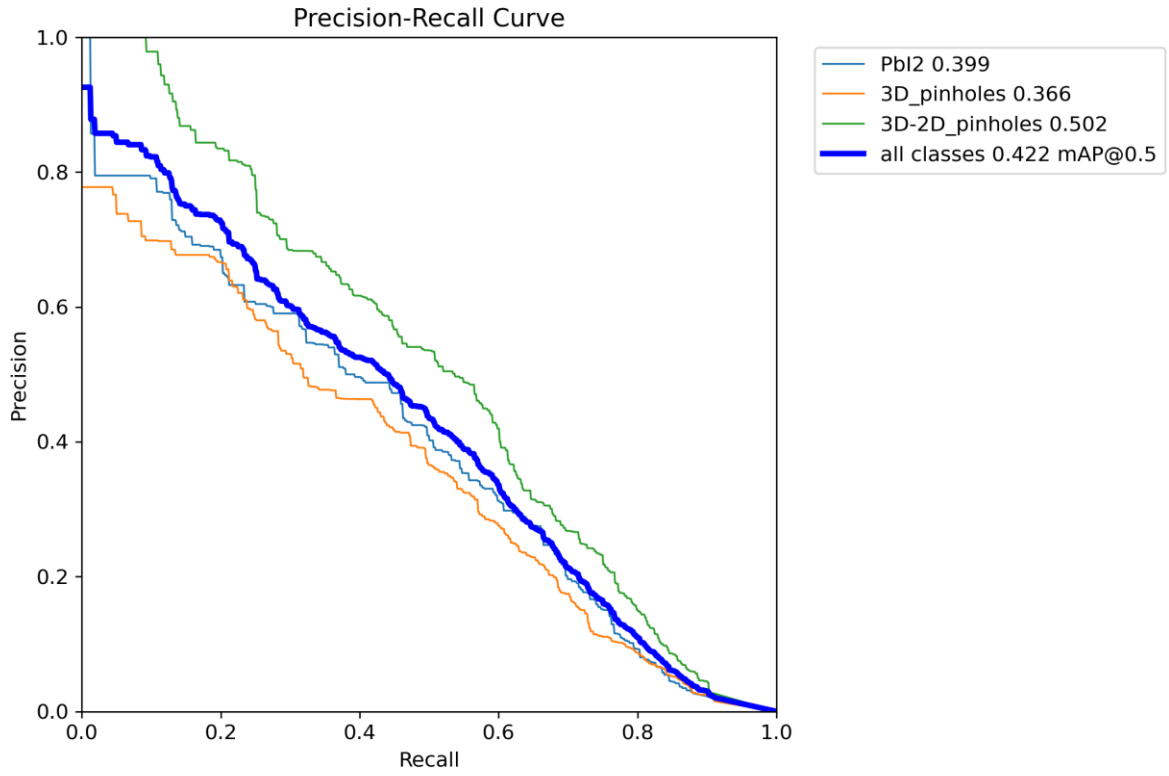


Figure 9. YOLO11m precision-recall curve artifact.

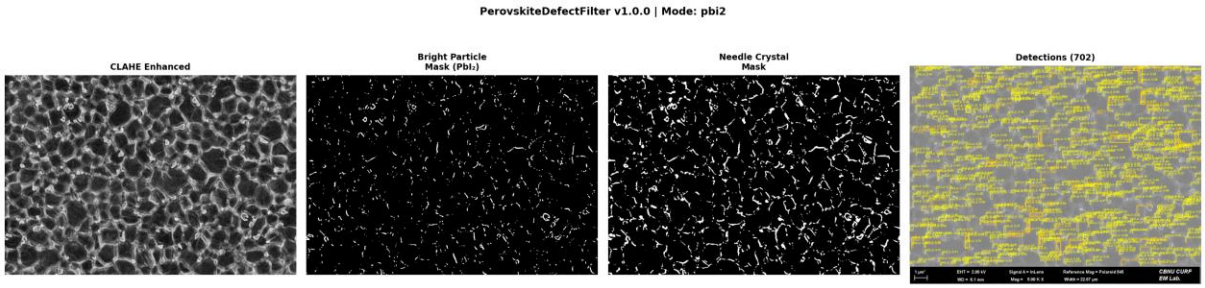


Figure 10. Classical Perovskite Defect Filter pipeline for a PbI2 sample.

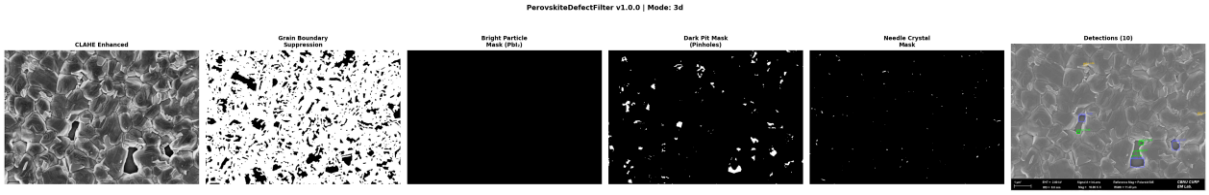


Figure 11. Classical Perovskite Defect Filter pipeline for a 3D pinhole sample.

7.2 Error Analysis

- Small-object limitation (smaller than the patch): may cause defects to be too small to generate strong localization signals.
- Texture ambiguity grains boundaries, and/or pores, and/or gradients in lighting can simulate a pinhole or PbI2 particles.
- Patch context loss: 224 224 patches cut out film's larger context.
- Path mismatch of app comparison: training metrics are src/runs/metrics, while the live Model Comparison tab searches runs/metrics.
- Manual-label dependency: model quality is strongly dependent on annotation consistency and coverage.
- Excessive label: background images are justified, yet an over abundance of empty label will taint the detector bias for predictions.

8. Pros, Cons, and Design Tradeoffs

The Tradeoffs relies in between Webapp UI, YOLO labels, 224 x 224 patches, OpenCV auto-labeling, SAM YOLO11m best result

Table 7: Critical Evaluation of System Design Decisions and Implementation Trade-offs.

Decision	Pros	Cons	Interpretation
Webapp UI	Fast to build, easy to demonstrate, no frontend stack.	State handling and long-running training can be fragile.	Good for academic/lab tool; production may need job queue and API backend.
YOLO labels	Simple text files, widely supported.	Bounding boxes are less precise than masks.	Appropriate first version; segmentation labels can be added later.
224 x 224 patches	Fits low VRAM and increases sample count.	May lose contextual cues and split defects.	Useful for RTX 3050-class hardware; test larger sizes later.
OpenCV auto-labeling	Transparent and fast; no GPU required.	Threshold tuning is image-dependent.	Excellent assistant for first-pass labels, not final ground truth.
SAM	Strong general segmentation prior.	Needs checkpoint and class assignment logic.	Promising for future mask labels.
YOLO11m best result	Best mAP50-95 among available runs.	Still moderate absolute performance.	Needs more data, higher resolution, augmentation, and per-class diagnosis.

9. Conclusion and Future Work

Reports are an end to end full SEM annotation and defect detection pipeline for perovskite solar cell images. We performed manual annotation, auto-labeling, classical image processing filters, trained with YOLO as well as worked on faster R-CNN infrastructure, performed detailed benchmark reporting and stored the results of experiments. We are using the state-of-the-art model which is YOLO11m but the midrange mAP50-95 score indicates we still have significant space for progress with respect to the current dataset and model options offered. The repository is useful because it moves past a single algorithm but supplies the entire loop an end user would need to execute in practice: annotate train compare, correct, and retrain.

9.1 Future Work

- Evaluate larger patch sizes such as 448 or 640 pixels for additional SEM context.
- Add SEM-specific augmentation: contrast jitter, blur/noise, rotation, scale, and synthetic defect insertion.
- Introduce segmentation models such as U-Net, Mask R-CNN, or YOLO segmentation for irregular pinholes and PbI2 regions.

References

[1] J. Guo Pan, L. Wang, and X. Cai, "Automated and Scalable SEM Image Analysis of Perovskite Solar Cell Materials via a Deep Segmentation Framework," arXiv:2509.26548, 2025. https://arxiv.org/abs/2509.26548

[2] "A multi-model deep learning framework for SEM-based defect detection in FAPbI3 Perovskite thin films," Scientific Reports, 2025. https://www.nature.com/articles/s41598-025-25848-x

[3] B. Dey et al., "Deep Learning based Defect classification and detection in SEM images: A Mask R-CNN approach," arXiv:2211.02185, 2022. https://arxiv.org/abs/2211.02185

[4] "A Review on Machine and Deep Learning for Semiconductor Defect Classification in Scanning Electron Microscope Images," Applied Sciences, vol. 11, no. 20, 9508, 2021. https://www.mdpi.com/2076-3417/11/20/9508

[5] "Investigation of Well-Defined Pinholes in TiO2 Electron Selective Layers Used in Planar Heterojunction Perovskite Solar Cells," Nanomaterials, 2020. https://www.mdpi.com/2079-4991/10/1/181

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," arXiv:1506.02640, 2015. https://arxiv.org/abs/1506.02640

[7] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," arXiv:1506.01497, 2015. https://arxiv.org/abs/1506.01497

[8] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," arXiv:1505.04597, 2015. https://arxiv.org/abs/1505.04597

[9] A. Kirillov et al., "Segment Anything," arXiv:2304.02643, 2023. https://arxiv.org/abs/2304.02643

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, 2016.

[11] T.-Y. Lin et al., "Feature Pyramid Networks for Object Detection," CVPR, 2017.

[12] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal Loss for Dense Object Detection," ICCV, 2017.

[13] K. He, G. Gkioxari, P. Dollar, and R. Girshick, "Mask R-CNN," ICCV, 2017.

[14] J. Canny, "A Computational Approach to Edge Detection," IEEE TPAMI, vol. 8, no. 6, 1986.

[15] N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," IEEE Transactions on Systems, Man, and Cybernetics, 1979.

[16] K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization," Graphics Gems IV, 1994.

[17] J. Serra, Image Analysis and Mathematical Morphology, Academic Press, 1982.

[18] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[19] OpenCV Documentation, "CLAHE," 2026. https://docs.opencv.org/4.x/d2/d74/tutorial_js_histogram_equalization.html

[20] OpenCV Documentation, "More Morphology Transformations," 2026. https://docs.opencv.org/3.4/d3/dbe/tutorial_opening_closing_hats.html

[21] OpenCV Documentation, "Canny Edge Detector," 2026. https://docs.opencv.org/3.4/da/d5c/tutorial_canny_detector.html

[22] Webapp Documentation, "Webapp documentation," 2026. https://docs.Webapp.io/

[23] Ultralytics, "Mean Average Precision (mAP)," 2026. https://www.ultralytics.com/glossary/mean-average-precision-map

[24] Torchvision Documentation, "fasterrcnn_resnet50_fpn," 2026. https://docs.pytorch.org/vision/0.26/_modules/torchvision/models/detection/faster_rcnn.html

[25] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," NeurIPS, 2019.

[26] A. Kojima, K. Teshima, Y. Shirai, and T. Miyasaka, "Organometal Halide Perovskites as Visible-Light Sensitizers for Photovoltaic Cells," JACS, 2009.

[27] H.-S. Kim et al., "Lead Iodide Perovskite Sensitized All-Solid-State Submicron Thin Film Mesoscopic Solar Cell," Scientific Reports, 2012.

[28] M. M. Lee et al., "Efficient Hybrid Solar Cells Based on Meso-Superstructured Organometal Halide Perovskites," Science, 2012.

[29] G. Niu, X. Guo, and L. Wang, "Review of Recent Progress in Chemical Stability of Perovskite Solar Cells," Journal of Materials Chemistry A, 2015.
[30] M. Saliba et al., "Cesium-containing triple cation perovskite solar cells," Energy & Environmental Science, 2016.
[31] S. D. Stranks et al., "Electron-Hole Diffusion Lengths Exceeding 1 Micrometer in an Organometal Trihalide Perovskite Absorber," Science, 2013.
[32] D. W. deQuilettes et al., "Impact of microstructure on local carrier lifetime in perovskite solar cells," Science, 2015.
[33] W. S. Yang et al., "Iodide management in formamidinium-lead-halide-based perovskite layers for efficient solar cells," Science, 2017.
[34] H. J. Snaith, "Perovskites: The Emergence of a New Era for Low-Cost, High-Efficiency Solar Cells," Journal of Physical Chemistry Letters, 2013.
[35] W. Tress, "Perovskite Solar Cells on the Way to Their Radiative Efficiency Limit," Advanced Energy Materials, 2017.